

Practical : 1	Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort
Date : 01/07/2026	

Objective :- To implement five major sorting algorithms (Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quicksort) in C++ using arrays, and to evaluate their combined time and space complexity characteristics.

Program :-

```
#include <iostream>
using namespace std;
// Function to print array elements
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
// 1. Bubble Sort
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap elements
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
// 2. Selection Sort
```

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        // Swap minimal found element with current index
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

// 3. Insertion Sort
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// Merge helper function for Merge Sort
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[100], R[100]; // Temporary arrays (Assuming safe max size for lab input)
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

// 4. Merge Sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

// Partition helper function for Quicksort
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Choosing the last element as pivot
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
}
```

```
        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;
        return i + 1;
    }
// 5. Quicksort
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int originalArr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(originalArr) / sizeof(originalArr[0]);
    int arr[100]; // Active working copy array
    cout << "Original Array: ";
    printArray(originalArr, n);

    // Testing Bubble Sort
    for(int i=0; i<n; i++) arr[i] = originalArr[i];
    bubbleSort(arr, n);
    cout << "\n1. Bubble Sort Result: ";
    printArray(arr, n);

    // Testing Selection Sort
    for(int i=0; i<n; i++) arr[i] = originalArr[i];
    selectionSort(arr, n);
    cout << "2. Selection Sort Result: ";
    printArray(arr, n);

    // Testing Insertion Sort
    for(int i=0; i<n; i++) arr[i] = originalArr[i];
```

```
insertionSort(arr, n);  
cout << "3. Insertion Sort Result: ";  
printArray(arr, n);  
  
// Testing Merge Sort  
for(int i=0; i<n; i++) arr[i] = originalArr[i];  
mergeSort(arr, 0, n - 1);  
cout << "4. Merge Sort Result: ";  
printArray(arr, n);  
  
// Testing Quicksort  
for(int i=0; i<n; i++) arr[i] = originalArr[i];  
quickSort(arr, 0, n - 1);  
cout << "5. Quicksort Result: ";  
printArray(arr, n);  
  
return 0;  
}
```

Output :-

```
Original Array: 64 34 25 12 22 11 90  
  
1. Bubble Sort Result: 11 12 22 25 34 64 90  
2. Selection Sort Result: 11 12 22 25 34 64 90  
3. Insertion Sort Result: 11 12 22 25 34 64 90  
4. Merge Sort Result: 11 12 22 25 34 64 90  
5. Quicksort Result: 11 12 22 25 34 64 90
```

Analysis:-

The algorithms tested can be naturally grouped into two categories based on their design patterns: **Quadratic-time basic sorts** $O(n^2)$ and **Logarithmic-time advanced divide-and-conquer sorts** $O(n \log n)$

Time Complexity Summary

- **Bubble, Selection, and Insertion Sort** utilize nested loop structures. In average and worst-case setups, they complete roughly n^2 computational comparison loops, giving them an $O(n^2)$ quadratic rate of growth. Insertion and optimized Bubble Sort perform exceptionally well $O(n)$ if the dataset is already mostly sorted.
- **Merge Sort and Quicksort** split the data into smaller fractions continuously. This divide-and-conquer strategy dramatically cuts down operations to an $O(n \log n)$ average rate. Quicksort can degrade to $O(n^2)$ if the pivot choices consistently isolate single elements instead of balanced splits.

Sorting Algorithms	Time Complexity			Space Complexity
	Best Case	Average Case	Worst Case	Worst Case
Bubble Sort	$O(N)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Selection Sort	$\Omega(N^2)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Insertion Sort	$\Omega(N)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Quick Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N^2)$	$O(N)$
Merge Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N \log N)$	$O(N)$

Space Complexity Summary

- **Bubble, Selection, and Insertion Sort** require $O(1)$ space because they arrange elements completely in-place without generating extra lists.
- **Merge Sort** requires a linear auxiliary footprint of $O(n)$ memory to construct temporary arrays during active array blending operations.
- **Quicksort** uses an execution stack footprint of $O(\log n)$ on average to keep track of recursive partition boundaries.

Conclusion :-

This practical highlights the direct real-world trade-off between algorithmic simplicity and scalability:

1. **For Small Datasets:** Simple algorithms like **Insertion Sort** and **Bubble Sort** are optimal because they require negligible implementation complexity and execute completely in place ($O(1)$ auxiliary space).
2. **For Large Datasets:** Divide-and-conquer algorithms like **Merge Sort** and **Quicksort** are vastly superior because their $O(n \log n)$ runtime allows them to process massive files cleanly. **Merge Sort** guarantees reliable stability but requires a higher memory profile ($O(n)$), while **Quicksort** offers highly rapid in-place processing under typical runtime conditions.